

PRODUCT · HIGHER EDUCATION · AGENTIC AI

EduGrant AI

Solution Design Document

Developer-ready architecture, agent specs, and 60-day delivery plan for the AI Summit MVP.

DOCUMENT

v1.0 — EduGrant AI MVP

OWNER

Nisarg Kadam · Solution Architect · Engineering Team · RiseLogic AI

AUDIENCE

STATUS

APPROVED FOR DEVELOPMENT

DOCUMENT MAP

Contents

01	Executive Summary Why we're building this and what success looks like	03
02	Problem Statement Current pain, target outcomes, success metrics	04
03	MVP Scope What's in, what's out, the AI Summit demo arc	05
04	System Architecture End-to-end topology and component map	06
05	Agent Specifications Each agent: purpose, I/O, tools, prompt strategy	08
06	LangGraph State Machine State schema, nodes, edges, conditional routing	13
07	Data Models Pydantic schemas for application, decision, audit	15
08	API Contract FastAPI endpoints — student, admin, agent traces	17
09	Frontend Specification Three React surfaces: portal, dashboard, trace UI	18
10	Repository Structure Folder layout, modules, conventions	20
11	Local Development Setup Step-by-step from clone to first agent run	21
12	Deployment — Railway Services, env vars, networking, observability	22
13	Testing & Evals PromptFoo, unit tests, golden examples	23
14	Sprint Plan — 60 Days Four sprints, owners, definitions of done	24
15	Risks & Open Questions What could go wrong and what we don't know yet	25
16	AI Summit Demo Script The exact sequence we walk through on stage	26

01 — EXECUTIVE SUMMARY

What we're building, why, and when.

EduGrant AI is a multi-agent system that automates the financial aid and scholarship application pipeline end-to-end. This document is the engineering contract for the MVP we will demo at AI Summit and subsequently productize under Riselogic.ai.

What this document is

A developer-ready specification. Anyone joining the team should be able to read this document, set up the project locally within an hour, and start implementing assigned agents. No tribal knowledge required.

What we are building

- **4 production-grade agents** orchestrated via LangGraph (Triage, Document Intelligence, Eligibility Scoring, Communication) plus a Supervisor.
- **3 React surfaces:** a student application portal, an admin decision dashboard, and a live agent-trace visualization.
- **One LLM, one cloud, one stack:** GPT-4o, Railway, FastAPI, LangGraph — kept simple to ship in 60 days.
- **Production discipline:** Pydantic v2 schemas, PromptFoo evals, observability via LangSmith, container deployments.

Success criteria for the MVP

Demo readiness	Live walkthrough of full pipeline at AI Summit — application to decision in under 5 minutes.
Technical KPIs	>90% extraction accuracy on test pack of 50 applications. <30s end-to-end agent latency. Zero unhandled exceptions.
Business KPIs	Three follow-up enterprise conversations from AI Summit demo. One signed pilot LOI within 30 days post-event.
Engineering KPIs	100% agents covered by PromptFoo evals. Single-command local setup. CI green on main.

STRATEGIC CONTEXT

EduGrant AI is the first of three Riselogic.ai products. WelfareWise.ai (Tier-5 multi-agent) and SemiSight AI (semiconductor fab intelligence) follow. Architecture decisions made here set the reference pattern for the next two builds.

02 — PROBLEM STATEMENT

The work being eliminated.

University financial aid offices process thousands of applications per intake cycle. Each application is a folder of unstructured documents — transcripts, recommendation letters, income proofs, personal essays — that must be read, validated, scored, and decided on by a small human team within a tight calendar window.

Current pain — what we measured

Cycle time	3 weeks average from submission to decision. 6-8 weeks at peak intake.
Manual triage	~45 minutes per application for first-pass review. Most of it routine.
Inconsistency	Up to 30% variance in eligibility scores between reviewers on the same file.
Document chasing	~40% of applications need follow-up for missing information. Each round costs days.
Audit burden	Decision rationale is captured ad-hoc. Audits require reconstructing reviewer logic from emails.

Target outcomes — what EduGrant delivers

- **3-day decisions** on the happy path. Same-day for clean, complete applications.
- **80% reduction** in human review time per application. Reviewers see only flagged or borderline cases.
- **Consistent scoring** backed by transparent rubrics — every decision is reproducible and defensible.
- **Autonomous outreach** for missing information. Students get clear, specific requests within the hour.
- **Full audit trail**: every input, every agent step, every decision logged with provenance.

OUT OF SCOPE FOR THE MVP

Multi-language applications, integration with university SIS, role-based access control beyond admin/student, and disbursement workflows. All deferred to v2 — captured as backlog in §15.

03 — MVP SCOPE

Slim demo. Single scholarship. Happy path.

We are building the smallest possible system that tells a complete agentic story on stage. Every feature must justify its weight against the AI Summit demo. If it doesn't make the demo more compelling, it doesn't ship in the MVP.

✓ IN SCOPE	✗ OUT OF SCOPE (V2 BACKLOG)
• One scholarship type — Merit-Based Undergraduate. • Happy path — clean apps process end-to-end without human touch. • One missing-doc edge case — to demo agentic outreach. • 4 specialized agents + LangGraph supervisor. • 3 React surfaces — student portal, admin dashboard, live agent trace. • Synthetic test data — 50 generated applications, no PII. • Single LLM — OpenAI GPT-4o for all tasks. • Email outbound only — outreach via SendGrid sandbox. • Railway deployment — 3 services, single environment.	• Multiple scholarship types and complex eligibility matrices. • Real student PII and university SIS integration. • Multi-language application support. • RBAC beyond admin/student split. • Disbursement and bank integration. • Mobile-native experience (web responsive only). • SSO / SAML for institutional accounts. • Full multi-tenant isolation per university. • Compliance certifications (SOC2, FERPA). • Replacing the LLM with a fine-tuned local model.

- **LangSmith tracing** for every agent invocation.

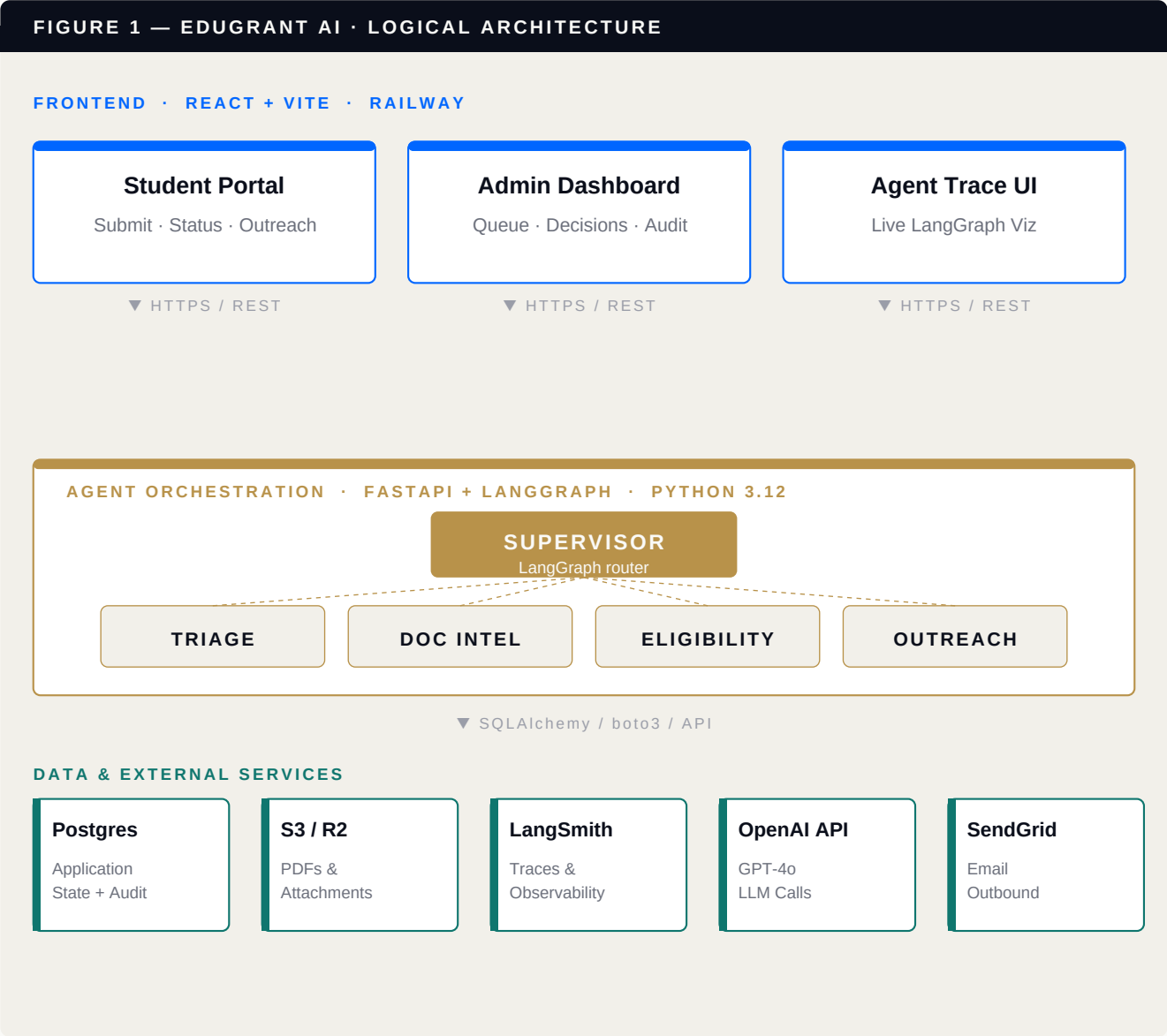
The AI Summit demo arc — 5 minutes on stage

- **0:00 — 0:30** · Open student portal. Submit a complete application with PDF attachments.
- **0:30 — 2:00** · Switch to live agent trace UI. Watch agents fire in real time — Triage routes, Document Intelligence extracts, Eligibility scores.
- **2:00 — 3:00** · Show admin dashboard with the auto-approved decision, full provenance, and audit timeline.
- **3:00 — 4:30** · Submit a second application with a missing transcript. Watch outreach agent draft and send the email autonomously.
- **4:30 — 5:00** · Close on the metrics: time saved, audit-readiness, zero manual touchpoints on happy path.

04 — SYSTEM ARCHITECTURE

End-to-end topology.

Three logical tiers — UI, Agent Orchestration, Data — deployed as separate Railway services. All communication is HTTPS. State persists in Postgres; documents live in object storage; agent telemetry streams to LangSmith.



04 — ARCHITECTURE (CONT'D)

Component responsibilities.

Service-by-service

Frontend (React)	Three SPAs served from one Railway service. Vite build. Communicates with Backend over REST. Renders LangGraph traces by polling /api/runs/{id}/events every 1s during active runs.
Backend (FastAPI)	Single FastAPI app. Hosts all REST endpoints. Triggers LangGraph runs as background tasks. Persists application + run state to Postgres. Streams agent events to LangSmith and back to UI.
LangGraph Orchestrator	Embedded inside the FastAPI process for the MVP (no separate worker). Defines the agent graph, executes via .ainvoke(), checkpoints state to Postgres for resume-on-HITL.
Postgres (Railway)	Stores: applications, attachments metadata, decisions, agent runs, agent events, audit logs. SQLAlchemy 2.0 async ORM. Alembic migrations.
S3-compatible storage	Cloudflare R2 for the MVP (zero egress fee). Holds raw PDFs and Document Intelligence outputs. Pre-signed URLs for upload/download — backend never proxies file bytes.
OpenAI API	GPT-4o for text reasoning, gpt-4o-mini for cheap tasks (triage, classification). Wrapped via LangChain ChatOpenAI. Rate-limit retry with exponential backoff.
LangSmith	Required, not optional. Every agent invocation traced with inputs, outputs, prompts, latency. Used for debugging, evals, and cost attribution.
SendGrid	Single transactional sender for outreach emails. Sandbox mode in MVP — emails captured but not actually sent. Real send-out is a config flag flipped post-pilot sign-off.

05.1 — AGENT SPECIFICATION

Triage Agent

AGENT

Purpose

The entry point. Classifies inbound applications by scholarship type, completeness, and priority. Routes the run to the right downstream path. Cheap and fast — it should never be the bottleneck.

INPUTS

- application_id** (UUID) from the FastAPI submission endpoint.
- raw_payload** (JSON) — student-provided form fields.
- attachment_manifest** — list of uploaded files with filenames and S3 keys.

OUTPUTS

- scholarship_type** — one of {merit_undergrad, ...}; MVP only handles merit_undergrad.
- completeness_score** (0.0–1.0) — heuristic check on required fields.
- routing_decision** — {proceed, request_missing_info, reject_invalid}.
- triage_reasoning** — short structured explanation, logged to audit.

Tools & integrations

- JSON-schema validator
- File-type sniffer
- LangGraph state writer

Prompt strategy

Few-shot prompt with 5 canonical examples (complete merit-undergrad app, missing transcript, missing essay, wrong scholarship type, fraudulent-looking submission). System prompt enforces JSON output via response_format={'type': 'json_schema'}. No chain-of-thought — Triage runs thousands of times, latency matters.

Model & fallbacks

Primary: gpt-4o-mini (fast, low cost)

Fallbacks: On API failure: rule-based completeness check using Pydantic validators only. Run still proceeds but flagged for manual review.

KPIs (eval gates)

- | | |
|--------------------|--|
| ■ Latency p95 | < 1.5 seconds per invocation |
| ■ Routing accuracy | ≥ 98% on golden test set (50 cases) |
| ■ False rejections | < 1% — never reject a complete application |

05.2 — AGENT SPECIFICATION

Document Intelligence Agent

AGENT

Purpose

Reads every PDF attached to the application. Extracts structured data — student info, transcript GPA, course list, recommender details, income figures. This is the heaviest agent and the one that demos most viscerally on stage.

INPUTS

- **application_id** + **attachment_manifest**.
- **scholarship_type** from Triage — determines which extraction schema to use.
- **S3 pre-signed URLs** for each attached PDF.

OUTPUTS

- **extracted_data** — Pydantic model populated with all required fields.
- **missing_fields** — list of required-but-not-found fields, with confidence scores.
- **document_quality_flags** — {scanned_low_quality, mismatched_name, expired_doc, ...}.
- **extraction_traces** — page-level provenance for every extracted value (for audit).

Tools & integrations

GPT-4o vision (PDF page rasterization)

PyMuPDF (fast text extraction first-pass)

Pydantic v2 schema enforcement

S3 reader

Prompt strategy

Two-pass extraction. **Pass 1:** PyMuPDF text-only — cheap and fast for digitally-born PDFs. **Pass 2:** if Pass 1 confidence is low or doc is scanned, fall back to GPT-4o vision per page. Per-field prompts use schema-as-instruction pattern: 'Extract the following Pydantic model from this document. Return JSON only.' Each field gets a confidence score; below 0.7 routes to outreach.

Model & fallbacks

Primary: gpt-4o (vision required)

Fallbacks: On vision API failure: text-only extraction proceeds, missing fields flagged for outreach. Never blocks the pipeline.

KPIs (eval gates)

■ Extraction F1	≥ 0.92 across all required fields
■ Latency per app	< 12 seconds end-to-end (5 PDFs avg)
■ Hallucination rate	< 0.5% — verified via PromptFoo factual eval

05.3 — AGENT SPECIFICATION

Eligibility Scoring Agent

AGENT

Purpose

The reasoning agent. Takes extracted application data, applies the scholarship rubric, produces a numerical score with full justification. This is where Riselogic.ai earns its 'replaces RPA' narrative — judgment, not rules.

INPUTS	OUTPUTS
extracted_data from Document Intelligence. scholarship_rubric — JSON config with weighted criteria (GPA, financial need, essay quality, references). historical_decisions — last 100 anonymized decisions for in-context calibration.	eligibility_score (0–100) with sub-scores per rubric dimension. recommendation — {auto_approve, auto_reject, human_review}. reasoning_chain — natural-language explanation tied to specific rubric criteria. confidence — model's stated confidence; below 0.85 always routes to human_review.

Tools & integrations

- Rubric loader (JSON)
- Few-shot retriever (historical decisions)
- Score validator

Prompt strategy

Chain-of-thought enabled here — quality matters more than latency. System prompt loads the rubric as an explicit scoring guide. User prompt provides extracted_data + 5 retrieved similar historical cases for calibration. Output is structured: per-criterion score, weighted total, natural-language justification. We **never** ask the LLM 'should we approve?' — we ask it to score, then deterministic code applies thresholds.

Model & fallbacks

Primary: gpt-4o (full reasoning)

Fallbacks: On API failure: route to human_review with all extracted data attached. Never auto-decides without the LLM.

KPIs (eval gates)

- | | |
|------------------------------|---|
| ■ Score vs. human | Mean absolute error < 8 points on golden set |
| ■ Auto-approve safety | 0% false-approve on rejected golden cases |
| ■ Reasoning quality | ≥ 90% reviewer agreement on rationale clarity |

05.4 — AGENT SPECIFICATION

Outreach Agent

AGENT

Purpose

Handles the missing-information edge case. When Document Intelligence flags missing fields, this agent drafts a personalized, polite, specific outreach email to the student and queues it for sending. Demoeed live on stage — the most relatable agentic moment.

INPUTS	OUTPUTS
• application_id + missing_fields from Document Intelligence. • student_contact from extracted_data. • scholarship_context — name, deadline, university branding tokens.	• email_draft — subject, body, suggested attachments-needed list. • send_decision — {queue_for_send, hold_for_review} (MVP: always queue). • resume_token — opaque ID the student includes in their reply to resume the LangGraph run.

Tools & integrations

- Email template renderer (Jinja2)
- SendGrid client
- LangGraph checkpoint resume

Prompt strategy

Constrained generation with a Jinja2 base template. LLM only fills in: (1) the personalized opening referencing student's name and program, (2) the precise list of missing items in plain English (not field names), (3) a closing paragraph matched to scholarship deadline urgency. Tone guardrails: warm, specific, never robotic. Subject line generated separately for A/B test readiness.

Model & fallbacks

- Primary:** gpt-4o-mini (cheap; templates do the heavy lifting)
- Fallbacks:** On API failure: send a fully-templated generic outreach. Better than no email.
- KPIs (eval gates)**

■ Reply rate	Track post-pilot — no MVP gate
■ Latency	< 3 seconds per draft
■ Tone audit	100% pass on PromptFoo tone evals (no robotic phrasing)

05.5 — AGENT SPECIFICATION

Supervisor (LangGraph orchestrator)

AGENT

Purpose

Not an LLM call — the LangGraph graph itself. Defines the state, routes between agents based on outputs, handles retries, manages HITL pauses, and persists checkpoints. The 'glue' that makes the four specialists into one system.

INPUTS

- **EduGrantState** (TypedDict) — the shared state object passed between all nodes.
- **start event** — triggered by FastAPI on application submission.

OUTPUTS

- **final state** — populated EduGrantState with decision, audit trail, all agent outputs.
- **events stream** — every node entry/exit pushed to LangSmith and the trace UI.
- **checkpoints** — Postgres-persisted state at every node boundary, enabling resume-on-HITL.

Tools & integrations

- LangGraph StateGraph
- Postgres checkpointer
- LangSmith tracer

Prompt strategy

No prompts — pure code. See §06 LangGraph State Machine for the full graph definition. Routing logic uses conditional edges based on completeness_score, missing_fields, and confidence thresholds.

Model & fallbacks

Primary: N/A (deterministic)

Fallbacks: On any agent failure: state captures the error, supervisor routes to a 'failed_run' terminal node that logs to audit and surfaces in admin dashboard. Never silent failures.

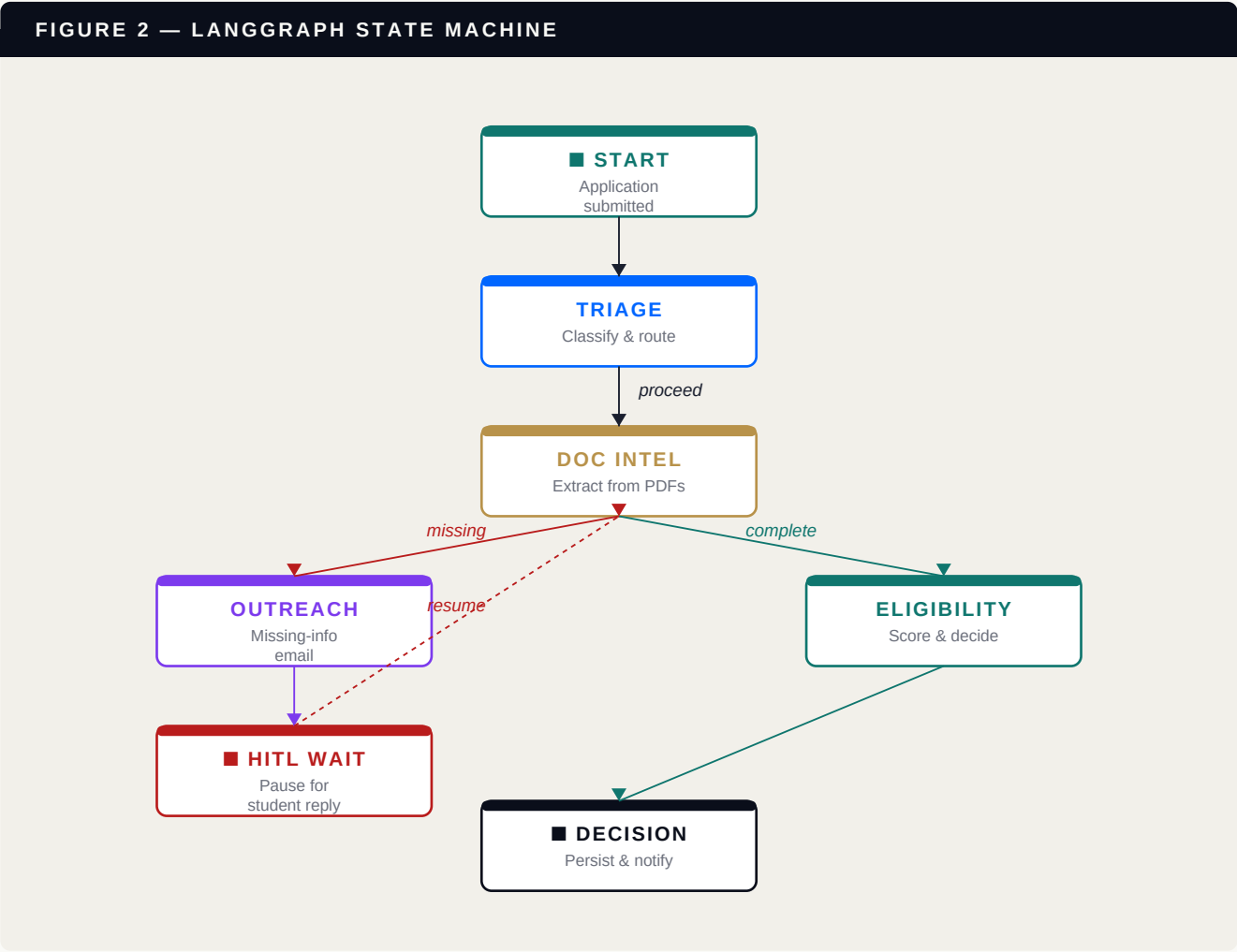
KPIs (eval gates)

- | | |
|----------------------|--|
| ■ Resume reliability | 100% successful resumes after HITL pause (golden test) |
| ■ Checkpoint write | < 200ms per node boundary |
| ■ Trace coverage | Every node emits start + end events to LangSmith |

06 — LANGGRAPH STATE MACHINE

The graph definition.

This is the canonical source of truth for orchestration logic. The diagram below maps 1:1 to the code in `edugrant/orchestrator/graph.py`. State transitions are deterministic; all LLM nondeterminism is contained inside agent nodes.



LEGEND — Solid = active edge · Dashed = resume after HITL · Conditional labels mark routing decisions

06 — LANGGRAPH (CONT'D)

The canonical graph.py

Reference implementation. Imports omitted for brevity — full file lives at **edugrant/orchestrator/graph.py** in the repo.

PYTHON

```
# edugrant/orchestrator/graph.py
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres import PostgresSaver

from edugrant.state import EduGrantState
from edugrant.agents import triage, doc_intel, eligibility, outreach

def route_after_triage(state: EduGrantState) -> str:
    if state["routing_decision"] == "reject_invalid":
        return "decision"
    return "doc_intel"

def route_after_doc_intel(state: EduGrantState) -> str:
    if state["missing_fields"]:
        return "outreach"
    return "eligibility"

def build_graph(checkpointer: PostgresSaver) -> StateGraph:
    g = StateGraph(EduGrantState)

    # Nodes
    g.add_node("triage", triage.run)
    g.add_node("doc_intel", doc_intel.run)
    g.add_node("outreach", outreach.run)
    g.add_node("eligibility", eligibility.run)
    g.add_node("decision", persist_decision) # deterministic node

    # Edges
    g.set_entry_point("triage")
    g.add_conditional_edges("triage", route_after_triage)
    g.add_conditional_edges("doc_intel", route_after_doc_intel)
    g.add_edge("outreach", END) # interrupt-before HITL_WAIT, resumed externally
    g.add_edge("eligibility", "decision")
    g.add_edge("decision", END)

    return g.compile(
        checkpointer=checkpointer,
        interrupt_after=["outreach"], # pauses for student reply
    )
```

Routing logic — plain English

- **After Triage:** if the application is fundamentally invalid (wrong scholarship, missing core fields), skip directly to Decision with `auto_reject`. Otherwise proceed to Document Intelligence.
- **After Document Intelligence:** if any required field is missing or low-confidence, route to Outreach. Otherwise proceed to Eligibility.
- **After Outreach:** the graph pauses (`interrupt_after`). LangGraph checkpoints state to Postgres. The graph resumes when the student replies — we re-run Document Intelligence with the new attachment, then re-evaluate the missing-fields check.
- **After Eligibility:** always proceed to Decision. The Eligibility agent's recommendation (`auto_approve` / `auto_reject` / `human_review`) is captured in state but Decision is a single deterministic node — no further branching.

07 — DATA MODELS

Pydantic v2 schemas.

All inter-agent state is typed. No untyped dicts ever cross a node boundary. The full schema lives at `edugrant/state/schemas.py`.

EduGrantState — the LangGraph state object

PYTHON

```
from typing import TypedDict, Annotated, Literal
from langgraph.graph.message import add_messages
from edugrant.state.schemas import (
    ApplicationPayload, ExtractedData, EligibilityResult, OutreachDraft
)

class EduGrantState(TypedDict):
    # Identity
    application_id: str
    run_id: str

    # Triage outputs
    scholarship_type: Literal["merit_undergrad"]
    completeness_score: float
    routing_decision: Literal["proceed", "request_missing_info", "reject_invalid"]
    triage_reasoning: str

    # Document Intelligence outputs
    extracted_data: ExtractedData | None
    missing_fields: list[str]
    document_quality_flags: list[str]

    # Outreach outputs (only set if routed)
    outreach_draft: OutreachDraft | None
    outreach_sent_at: str | None

    # Eligibility outputs
    eligibility_result: EligibilityResult | None

    # Final
    final_decision: Literal["approved", "rejected", "review"] | None
    audit_trail: Annotated[list[dict], add_messages]
```

ExtractedData — what Document Intelligence produces

PYTHON

```
from pydantic import BaseModel, Field, condecimal
from datetime import date

class StudentInfo(BaseModel):
    full_name: str
    email: str
    date_of_birth: date
    nationality: str
    student_id: str | None = None

class TranscriptInfo(BaseModel):
    institution: str
    gpa: condecimal(ge=0, le=4) = Field(..., description="On 4.0 scale")
    courses_completed: int
    cumulative_gpa: float = Field(..., ge=0, le=1)

class ExtractedData(BaseModel):
```

07 — DATA MODELS (CONT'D)

Postgres tables.

Five tables. SQLAlchemy 2.0 async ORM. Alembic-managed migrations. All tables have **created_at** and **updated_at** with TZ-aware timestamps.

applications	PK: application_id (UUID). Holds raw payload, scholarship_type, student_email, status. One row per submission.
attachments	PK: attachment_id (UUID). FK: application_id. Holds S3 key, mime_type, original_filename, uploaded_at. No file bytes — they live in R2.
agent_runs	PK: run_id (UUID). FK: application_id. Holds graph_version, started_at, completed_at, final_decision, total_cost_usd. One row per LangGraph run (a single application can have multiple runs after HITL resume).
agent_events	PK: event_id (BIGSERIAL). FK: run_id. Holds node_name, event_type {start, end, error}, input_summary (JSONB), output_summary (JSONB), latency_ms, model_used, tokens_in, tokens_out. One row per node entry/exit. This is the audit goldmine.
decisions	PK: decision_id (UUID). FK: application_id, run_id. Holds final_decision, eligibility_score (JSONB), reasoning_text, decided_at, decided_by ('agent' or human user_id). Immutable — corrections create a new row.

MIGRATION DISCIPLINE

Every schema change goes through Alembic. CI enforces: no migration in main without a paired rollback test. Production migrations run in a dedicated Railway one-off task — never on app boot.

08 — API CONTRACT

FastAPI endpoints.

All endpoints are async, return JSON, and emit OpenAPI specs at **/docs**. Authentication: HTTP Bearer for admin endpoints; signed pre-shared tokens for student portal.

POST /api/applications

Student submits a new application. Body: ApplicationPayload + signed S3 upload URLs already used. Response: {application_id, run_id}. Triggers a new LangGraph run as a FastAPI BackgroundTask.

GET /api/applications/{id}

Student fetches their own application status. Auth: signed token. Returns: status, current_node, last_event_at, missing_fields (if any).

POST /api/applications/{id}/reply

Student responds to outreach with new attachments. Resumes the paused LangGraph run. Response: {run_id, resumed: true}.

GET /api/admin/queue

Admin lists all applications by status. Auth: Bearer. Filters: status, scholarship_type, date_range. Returns paginated list with eligibility_score and recommendation.

GET /api/admin/applications/{id}

Admin views full application detail — extracted data, all attachments, complete decision audit trail.

POST /api/admin/applications/{id}/override

Admin overrides an agent decision. Creates a new immutable row in decisions. Logged to audit.

GET /api/runs/{run_id}/events

Live agent trace UI polls this every 1s during active runs. Returns ordered list of agent_events. Will migrate to SSE post-MVP.

GET /api/runs/{run_id}/state

Returns the current EduGrantState as JSON. Used by the trace UI to render the LangGraph visualization with current node highlighted.

09 — FRONTEND SPECIFICATION

Three React surfaces.

All UI is React 18 + TypeScript + Vite + Tailwind. Component library: shadcn/ui. Charts: Recharts. Animations: Framer Motion. Built by Claude Code from prompted specs — see §11 for the prompt-as-spec workflow.

Student Portal	/apply, /status/:id
Two screens. Apply: a single-page form with file upload (drag-drop, multi-file, progress). On submit, redirects to /status/:id which shows a clean status timeline ('Received → Reviewing → Action needed → Decided') with the current step highlighted. Light, minimal, university-feeling.	
Admin Dashboard	/admin/queue, /admin/applications/:id
Queue view: filterable table of all applications, color-coded by recommendation, sortable by score. Detail view: split-pane — left shows extracted data + attachments, right shows full audit timeline with every agent step expanded. Decision controls: Approve / Reject / Override with reason.	
Live Agent Trace UI	/trace/:run_id
The demo's centerpiece. Renders the LangGraph visually as nodes-and-edges using react-flow. Active node pulses blue. Edge animates as state moves. Side panel shows current EduGrantState as live-updating JSON. Bottom drawer streams agent_events with timestamps, latencies, and token costs.	

09 — FRONTEND (CONT'D)

Design tokens & build workflow.

Design tokens

Brand color	#0066FF (Riselogic blue) — primary actions, focus states, agent active pulse
Surface	#FAFAF7 (off-white) — main background, premium publication feel
Typography	Inter for UI, JetBrains Mono for code/state JSON
Spacing	Tailwind defaults — 4px base. No custom spacings unless absolutely needed
Components	shadcn/ui primitives only — no custom components for the MVP
Motion	Framer Motion easings: easeOut for entries, easeInOut for state transitions
Iconography	lucide-react. No custom icons in MVP

Claude Code prompt-as-spec workflow

We build the React surfaces by prompting Claude Code with structured specs. Each component has a markdown spec checked into the repo at `frontend/specs/{ComponentName}.md`. Claude Code reads the spec and generates the component. Process:

- 1. Architect writes the spec — props, behavior, edge cases, design tokens used.
- 2. Developer runs claude in `frontend/` and points it at the spec.
- 3. Generated component lands in `src/components/` with co-located test.
- 4. Developer reviews, runs `npm run test` and `npm run typecheck`, commits.
- 5. Spec stays in repo as living documentation. Re-prompts use the spec as ground truth.

WHY PROMPT-AS-SPEC, NOT VIBE-CODING

Specs are reviewable, versionable, and re-runnable. When we add a new scholarship type or tweak the trace UI for a new audience, we update the spec — not the component. Claude Code regenerates. This is how we'll keep three productized surfaces consistent across all three Riselogic.ai products without a UI engineer per product.

10 — REPOSITORY STRUCTURE

Where everything lives.

Monorepo, two top-level apps (backend, frontend), shared schemas in a third package. uv for Python deps. pnpm for Node. Single Dockerfile per service.

PYTHON

```

edugrant-ai/
  README.md
  docker-compose.yml      # local dev: postgres + minio + langsmith
  .env.example
  pyproject.toml          # uv workspace root
  backend/                # FastAPI + LangGraph
    pyproject.toml
    Dockerfile
    alembic/              # migrations
    edugrant/
      api/                # FastAPI routes
        applications.py
        admin.py
        runs.py
      orchestrator/
        graph.py          # the LangGraph definition
        checkpointer.py
        agents/
          triage.py
          doc_intel.py
          eligibility.py
          outreach.py
        state/
          schemas.py      # Pydantic v2
          db.py           # SQLAlchemy
        tools/            # shared tools (S3, sendgrid, etc.)
        config.py
      tests/
        unit/
        integration/
        evals/            # PromptFoo configs
  frontend/               # React + Vite + Tailwind
    package.json
    Dockerfile
    specs/                # prompt-as-spec markdown files
    src/
      apps/
        student/
        admin/
        trace/
        components/      # shadcn/ui based
        lib/
      tests/
  shared/
    openapi.yaml          # generated from FastAPI; consumed by frontend

```

11 — LOCAL DEVELOPMENT

From clone to first agent run.

Target: any developer with a fresh laptop should be running the full stack locally in **under 30 minutes**. If your setup takes longer, the docs are wrong — file an issue.

Prerequisites

- Python 3.12+ (managed via uv)
- Node 20+ and pnpm
- Docker Desktop (for postgres + minio via docker-compose)
- An OpenAI API key (request from Ajay — see Tech Stack Proposal)
- A LangSmith project + API key (free tier sufficient for MVP)
- VS Code + recommended extensions (.vscode/extensions.json checked in)

Setup commands

PYTHON

```
# 1. Clone & enter
git clone git@github.com:riselogic-ai/edugrant-ai.git
cd edugrant-ai

# 2. Copy env file & fill in your keys
cp .env.example .env
# edit .env: OPENAI_API_KEY, LANGCHAIN_API_KEY, etc.

# 3. Install Python deps (uv handles workspace)
uv sync

# 4. Install frontend deps
cd frontend && pnpm install && cd ..

# 5. Boot local infra (postgres + minio)
docker-compose up -d

# 6. Run migrations
cd backend && uv run alembic upgrade head && cd ..

# 7. Seed test data (50 synthetic applications)
uv run python -m edugrant.scripts.seed_test_data

# 8. Start backend + frontend (two terminals)
# Terminal A:
uv run uvicorn edugrant.main:app --reload --port 8000
# Terminal B:
cd frontend && pnpm dev    # serves on :5173

# 9. Trigger your first agent run
curl -X POST http://localhost:8000/api/applications \
  -H "Content-Type: application/json" \
  -d @backend/tests/fixtures/sample_application.json

# 10. Watch it live in the trace UI:
# http://localhost:5173/trace/<run_id>
```

12 — DEPLOYMENT

Railway services & networking.

Three Railway services in one project. Postgres is a managed Railway plugin. R2 (Cloudflare object storage) is external. Total expected monthly cost: ~\$45 for the MVP (see Tech Stack Proposal for line items).

Service map

edugrant-backend	FastAPI + LangGraph. Dockerfile from backend/ . Port 8000. Health check on /health. Auto-deploy on main branch push.
edugrant-frontend	Vite static build served by nginx. Dockerfile from frontend/ . Port 80. Single-page-app fallback configured.
postgres (plugin)	Railway-managed Postgres 16. Connection string injected as DATABASE_URL into backend service.
Cloudflare R2 (external)	S3-compatible bucket: edugrant-prod-attachments . Pre-signed URLs only — no public access.

Required environment variables

```
PYTHON
# Railway service: edugrant-backend
OPENAI_API_KEY=sk-...
LANGCHAIN_API_KEY=ls___...
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=edugrant-prod

DATABASE_URL=${{ Postgres.DATABASE_URL }}      # Railway plugin reference

R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET=edugrant-prod-attachments

SENDGRID_API_KEY=SG...
SENDGRID_SANDBOX_MODE=true                    # MVP: don't actually send
```

```
ENV=production
LOG_LEVEL=INFO
RISELOGIC_INTEGRATION_BASE_URL=https://riselogic.ai
```

13 — TESTING & EVALS

Three layers, one CI gate.

Agent code without evals is a liability. Every agent ships with PromptFoo evals; every endpoint ships with integration tests; every Pydantic model ships with unit tests. CI fails if any layer drops below threshold.

Unit tests (pytest)

What: Pydantic validators, deterministic helpers, routing logic.

Where: backend/tests/unit/

Gate: 100% pass, >80% coverage on non-LLM code.

Integration tests (pytest + httpx)

What: End-to-end agent runs against a mocked OpenAI (deterministic LLM stub).

Where: backend/tests/integration/

Gate: 100% pass on the 5 golden scenarios (clean app, missing transcript, low GPA, borderline score, fraudulent submission).

LLM evals (PromptFoo)

What: Real LLM calls against curated test sets per agent. Measures extraction accuracy, scoring calibration, tone of outreach emails, hallucination rate.

Where: backend/tests/evals/

Gate: per-agent KPIs from §05 must pass. Run nightly + on prompt changes.

GOLDEN TEST SET

We build a fixed set of 50 synthetic applications covering all routing paths. Every PR runs against this set. Drift in extraction or scoring is caught before it reaches main. The set lives at **backend/tests/fixtures/golden/**.

14 — SPRINT PLAN

60 days. Four sprints. AI Summit ready.

Two-week sprints. Demo on Friday afternoon of sprint close. Definition of done is binary — code merged, tests passing, deployed to Railway preview.

Sprint 1 · Days 1-14

Foundations

Scope: Repo bootstrap. Local dev environment working. Postgres schema + migrations. FastAPI skeleton with one health-check endpoint. LangGraph 'hello world' graph with one stub agent. CI pipeline green.

Done when: All developers can run the stack locally. First commit in main triggers a successful Railway preview deploy.

Sprint 2 · Days 15-28

Agents · Triage + Document Intelligence

Scope: Build Triage agent end-to-end with PromptFoo evals. Build Document Intelligence agent with text-only extraction first, vision fallback second. Wire both into the LangGraph graph. Build /api/applications POST endpoint.

Done when: Submit a synthetic application via curl, watch Triage + Doc Intel run, see extracted data persisted in Postgres. PromptFoo evals passing per \$05 KPIs.

Sprint 3 · Days 29-42

Agents · Eligibility + Outreach + HITL

Scope: Build Eligibility scoring agent with rubric loader. Build Outreach agent with SendGrid sandbox. Wire HITL pause/resume via interrupt_after. Frontend: student portal + admin queue (basic).

Done when: Full happy path runs end-to-end on a clean application. Missing-transcript scenario pauses, outreach email captured in SendGrid sandbox, resume on synthetic reply.

Sprint 4 · Days 43-56

Polish · Trace UI · Demo prep

Scope: Build the live agent trace UI (the demo's centerpiece). Polish admin dashboard. Write demo script. Run dress rehearsals. Fix what breaks. Buffer for the unexpected (4 days reserved).

Done when: Three flawless demo run-throughs in front of internal audience. Trace UI looks unmistakably premium. AI Summit talk track timed at 5:00 ± 15s.

15 — RISKS & OPEN QUESTIONS

What could go wrong.

Risks ordered by likelihood × impact. Mitigation owner identified for each. Reviewed at every sprint demo.

HIGH Document Intelligence accuracy on scanned PDFs

Vision-based extraction may hallucinate on poor-quality scans. Mitigation: confidence threshold + auto-route to outreach for ambiguous fields. Owner: Doc Intel agent dev.

HIGH Demo-day live API failures (OpenAI outage, Railway hiccup)

Have a recorded video demo ready as backup. Cache one full happy-path run in a static fixture for offline replay if needed. Owner: Architect.

MED Eligibility scoring drift after prompt tweaks

PromptFoo evals run on every PR. Score variance >10% on golden set fails CI. Owner: Eligibility agent dev.

MED LangGraph HITL resume bugs

Most novel piece in the stack. Dedicated integration test running every PR. Manual smoke test before each demo. Owner: Orchestrator dev.

LOW Railway free-tier quota exceeded

Move to paid tier in Sprint 1 once team is real. Cost is in Tech Stack Proposal. Owner: Architect.

LOW Trace UI jank under fast event streams

Throttle UI updates to 4Hz max. Owner: Frontend dev.

Open questions for the architect

- Do we want a separate worker process for LangGraph, or keep it in-process with FastAPI for the MVP? *Current decision: in-process.*
- Which Postgres checkpointer — official LangGraph one, or build a thin custom wrapper for richer audit fields? *Current decision: official, augment via DB triggers.*
- Should the trace UI use react-flow or build custom SVG? *Current decision: react-flow for speed.*

16 — AI SUMMIT DEMO SCRIPT

Five minutes. Three windows. Zero stumbles.

The exact sequence. Rehearse this until you can do it in your sleep. Window setup before going on stage: three browser tabs, pre-arranged in this order.

Pre-stage setup

- Tab 1 — Student Portal (/apply) — empty form ready.
- Tab 2 — Live Trace UI (/trace) — empty, will populate when run starts.
- Tab 3 — Admin Dashboard (/admin/queue) — showing prior demo runs for context.
- Test microphone, screen share, confirm Railway services healthy via /health endpoints.

On-stage script

0:00 — 0:30 OPEN

Say: 'In a typical university financial aid office, this application would sit in someone's queue for three weeks. Watch what happens when agents handle it.' Switch to Tab 1. Submit a pre-filled application. Click Submit.

0:30 — 2:00 AGENT TRACE

Switch to Tab 2. **Say:** 'This is the agent network that just woke up.' Point at Triage as it fires — narrate it routing the application as a merit-undergrad happy path. Point at Document Intelligence as it processes the PDFs — call out the live extraction. Point at Eligibility as it scores. Pause as state JSON updates in real time.

2:00 — 3:00 DECISION

Switch to Tab 3. **Say:** 'Three minutes ago this application didn't exist. Now it has a decision, a score, and a complete audit trail.' Open the application detail. Show extracted data, expand the audit timeline. **Punch line:** 'Every step is logged. Every field traceable to a PDF page. This is what audit-ready actually looks like.'

3:00 — 4:30 EDGE CASE

Back to Tab 1. **Say:** 'Now let's try one with a missing transcript.' Submit second application. Switch to Tab 2. Watch the graph route to Outreach. Switch to SendGrid sandbox tab. Show the email the agent drafted. **Punch line:** 'Notice that's not a template. The agent wrote that — including the specific list of documents missing.'

4:30 — 5:00 CLOSE

Show the metrics slide: 80% time reduction, 3-day decisions, 100% audit-ready, infinite concurrent applications. **Close with:** 'This isn't an RPA bot. It's a system that handles judgment. We built it in 60 days. It's running in production at riselogic.ai. Three more like it are coming.'